

Welche Sprache wird von dem folgenden regulären Ausdruck beschrieben?

$a+a(b)^*a$

Der Regex beschreibt eine Sprache:

- die mit beliebig vielen a's anfängt
- danach ein a besitzt, (danach ein a oder mehrere a's und/oder ein b, diese Abfolge darf beliebig oft wiederholt werden)
- mit einem a endet

A1.2 Bezeichner in Programmiersprachen

Regex für die Bezeichner der Programmiersprache:

$^[a-zA-Z](_[a-zA-Z\d])^+$$

Input

```
_a_aa  
aae_aasfegwegnk  
aaa_  
a__a_a  
feffasjweiofg  
a_a_a  
a_a_a  
a__a__a  
aa  
vv  
cc|  
bb
```

rec: 83 14

Output

```
_a_aa  
aae_aasfegwegnk  
aaa_  
a__a_a  
feffasjweiofg  
a_a_a  
a_a_a  
a__a__a  
aa  
vv  
cc  
bb
```

- Start mit Buchstaben
- danach 1 oder mehr Zeichen (Buchstaben, Zahlen und Unterstrich), damit der Variablenname mind. 2 Zeichen groß ist, der Unterstrich muss vor den Buchstaben dran kommen, damit es nicht mit ihm endet.

zwei Beispiele:

pTest

- start mit p, das ist gültig: $^{\wedge}[a-zA-Z]$
- danach beliebig viele Buchstaben, Zahlen und Unterstriche, aber mindestens einer
"Test": $(_*[a-zA-Z\d])+\$$
- der Name endet nicht mit einem Unterstrich.
 - somit ist der Bezeichner gültig.

V_

- Start mit V, gültig
- danach beliebig viele Buchstaben, Zahlen und Unterstriche (und mindestens einer):
gültig
- Ende mit einem Unterstrich: der Bezeichner ist ungültig.

DFAs

Tupel des DFA der Programmiersprache:

(Zustände, Alphabet, Übergangsfunktion, Startzustand, Endzustände)

Zustände:

q0: ein Startzeichen [V|v|p|P] (Startzustand)

q1: nach dem Startzeichen ein Zeichen aus unserem Alphabet

unterstrichTest: bei Eingabe eines Unterstrichs

accept: unsere Eingabe ist gültig (Endzustand)

reject: eingabe ist ungültig

Alphabet von Eingabesymbolen:

0-9, a-z, A-Z, _

Übergangsfunktionen zu Zuständen:

Zustand	Eingabe	nächster Zustand
q0	a-z, A-Z	q1
q0	sonst	reject

q1	a-z, A-Z, 0-9	accept
q1	–	reject
unterstrichTest	a-z, A-Z, 0-9	q2
unterstrichTest	–	reject
accept	a-z, A-Z, 0-9	accept
accept	–	unterstrichTest

Startzustand:

- q0

Endzustände (akzeptierende Zustände):

- accept

2 Beispiele:**pTest**

q0 -> p -> q1
 q1 -> T -> accept
 accept -> e -> accept
 accept -> s -> accept
 accept -> t -> accept

accept ist der akzeptierende Zustand: Eingabe ist gültig

pNein_

q0 -> p -> q1
 q1 -> N -> accept
 accept -> e -> accept
 accept -> i -> accept
 accept -> n -> accept
 accept -> _ -> reject

reject ist kein Akzeptierender Zustand: Eingabe ist ungültig

reguläre Grammatik

$G = (N, T, P, S)$

Nichtterminale = (S, A, B) (Startsymbol, A für ein weiteres Zeichen, B Zustand wird nur erreicht, falls A ein Unterstrich ist)

Terminale (alle Zeichen) = a-z, A-Z, 0-9, _

richtig:

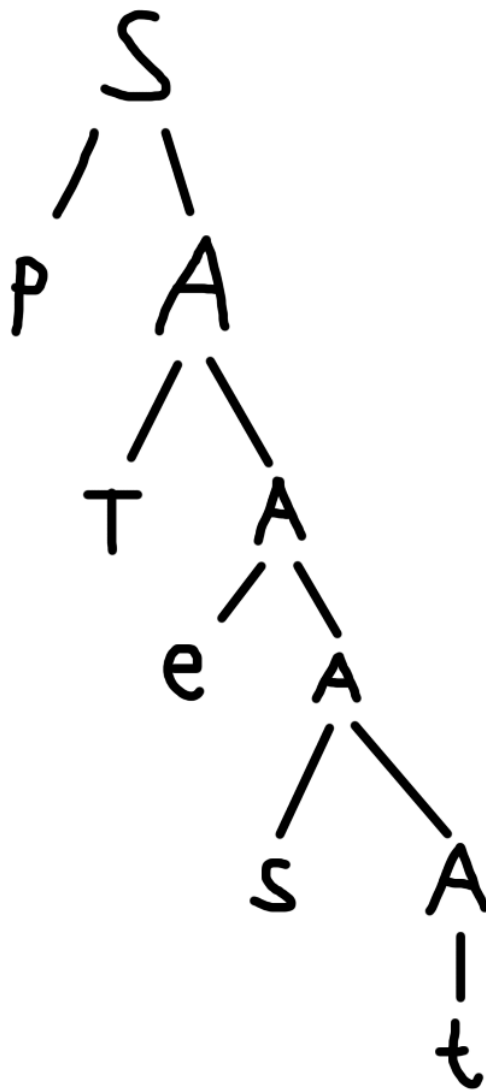
$S \rightarrow a A \mid b A \mid \dots \mid z A \mid A A \mid B A \mid \dots \mid Z A$

$A \rightarrow a A \mid b A \mid \dots \mid z A \mid A A \mid B A \mid \dots \mid Z A \mid 0 A \mid 1 A \mid \dots \mid 9 A \mid _ B \mid \epsilon$

$B \rightarrow a A \mid b A \mid \dots \mid z A \mid A A \mid B A \mid \dots \mid Z A \mid 0 A \mid 1 A \mid \dots \mid 9 A \mid _ B$

Beispiele mit 2 Bezeichnern

pTest



pTe_st

